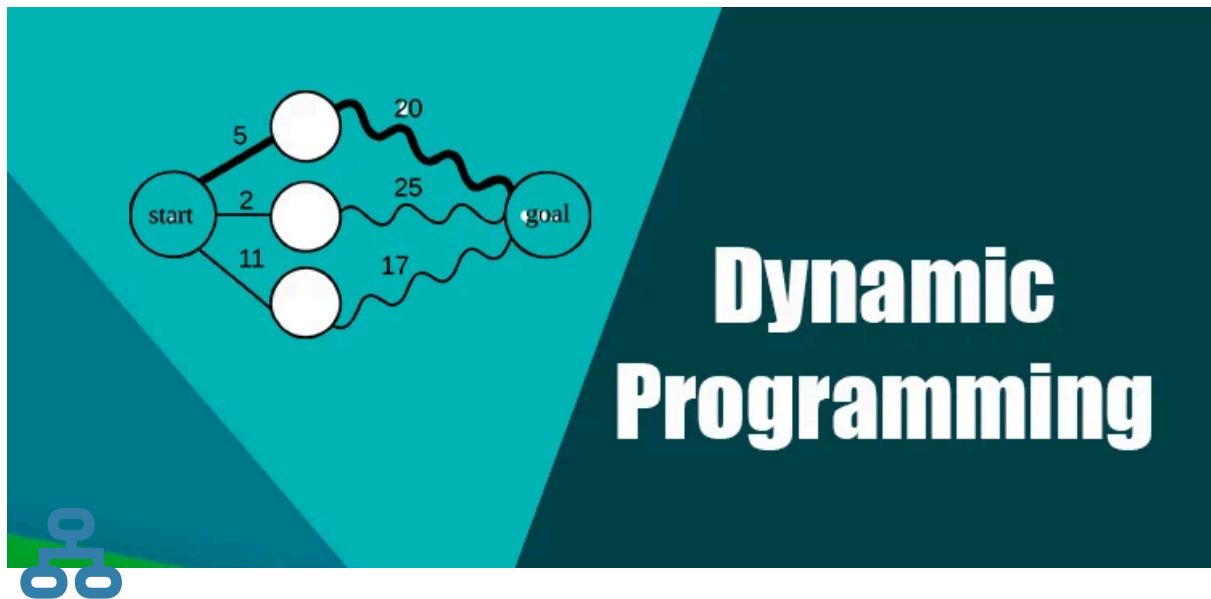




4. Dynamic Programming

"Those who fail to learn from history are condemned to repeat it."

Previous Section:

 [3. Backtracking](#)

Contents:

[1. What is Dynamic Programming?](#)

[Dynamic Programming vs Divide and Conquer](#)

[Dynamic Programming vs Recursion](#)

[3. Approaches of Dynamic Programming](#)

[Top-Down Approach \(Memoization\)](#)

[Bottom-Up Approach \(Tabulation\)](#)

[4. Characteristic of Dynamic Programming](#)

[Optimal Substructure](#)

[Overlapping Subproblems](#)

[5. Applications of Dynamic Programming](#)

[6. Advantages of Dynamic Programming](#)

[7. Disadvantages of Dynamic Programming](#)

[Summary](#)

[References](#)

Some problems are difficult not because the solution itself is impossible, but because the same work keeps repeating again and again.

Imagine trying to climb a staircase while forgetting every step you already calculated. You keep recomputing the same possibilities repeatedly. That is exactly what happens in many recursive problems. Dynamic Programming (DP) is an approach designed to avoid this waste.

Instead of solving the same subproblem over and over, Dynamic Programming solves it once, stores the result, and reuses it whenever needed. This makes DP one of the most powerful techniques for solving optimization problems efficiently.

At the heart of Dynamic Programming lies an important idea called the **Principle of Optimality**: *An optimal solution can be constructed from optimal solutions of its smaller subproblems.*

In other words, if the smaller parts are solved correctly, they can contribute to solving the larger problem optimally.

And that simple idea becomes the foundation of many famous algorithms in Computer Science.

1. What is Dynamic Programming?

Dynamic Programming (or DP) is an algorithmic technique that solves a complex problem by breaking it down into smaller sub-problems, where the solutions to these smaller problems contribute to solving the overall problem.

At first glance, this sounds very similar to Divide and Conquer because both approaches divide a large problem into smaller ones. But the difference between them is extremely important.

- In Divide and Conquer, the smaller problems are solved independently, and their results are combined afterward. The subproblems usually do not overlap.

- Dynamic Programming works differently. In DP, many of the smaller problems are repeated again and again. These are called **overlapping subproblems**. Instead of recomputing them repeatedly, DP solves each subproblem only once, stores the result, and reuses it later whenever needed. This avoids redundant computations and significantly improves efficiency.

So rather than treating every recursive call as completely separate, Dynamic Programming remembers previous work and uses it as future reference for solving larger problems.

Think of it like this:

- Divide and Conquer → Solve independently, then combine.
- Dynamic Programming → Solve once, remember, and reuse.

This is why DP is often far more efficient for optimization problems.

Dynamic Programming vs Divide and Conquer

Dynamic Programming is generally more effective than Divide and Conquer and Backtracking when problems contain overlapping computations. Instead of exploring every possibility repeatedly, DP selectively keeps only the useful computations that contribute toward the final solution. This greatly reduces time complexity.

The key distinction is:

- In Divide and Conquer, smaller solutions are only used to build bigger solutions.
- In Dynamic Programming, previous solutions remain important throughout the process and may be reused many times.

Sometimes DP uses all previously computed states, such as in: All-Pairs Shortest Path algorithms. And sometimes it selectively chooses only certain paths or states, such as: Multi-Stage Graph problems

So Dynamic Programming is not simply "divide and solve". It is more like:

| ***"Solve carefully, preserve useful knowledge, and build upon it efficiently."***

Dynamic Programming vs Recursion

Dynamic Programming is also closely related to Recursion. In fact, many DP algorithms begin as recursive solutions.

A recursive function solves a problem by calling itself on smaller inputs. However, primitive recursion often becomes inefficient because the same inputs are recomputed repeatedly. For example, a recursive algorithm may solve the exact same subproblem dozens, hundreds, or even millions of times.

Dynamic Programming optimizes this process. Instead of recomputing repeated calls, DP stores previously computed results and reuses them whenever the same input appears again. So DP is not a replacement for recursion—it is an optimization technique for recursive problems.

However, not every recursive problem can become a Dynamic Programming problem. For Dynamic Programming to work, the problem must contain overlapping subproblems. If the subproblems are independent and do not repeat, then the problem is usually solved using Divide and Conquer instead.

For example: Merge Sort and Quick Sort. These algorithms are recursive, but they are not considered Dynamic Programming algorithms because their subproblems do not significantly overlap.

So the important idea becomes:

- Recursion divides problems into smaller calls.
- Dynamic Programming improves recursion by avoiding repeated work.
- Divide and Conquer handles independent subproblems.
- DP handles overlapping subproblems efficiently.

And that difference is what makes Dynamic Programming one of the most powerful optimization techniques in Computer Science.

3. Approaches of Dynamic Programming

Dynamic Programming can generally be implemented using two major approaches: Top-Down Approach and Bottom-Up Approach

At first glance, both approaches solve the same problem. Both rely on the same recurrence relation. Both aim to avoid repeated computations. But the philosophy behind them is completely different.

Top-Down Approach (Memoization)

The Top-Down approach, commonly called **Memoization**, begins from the final problem and continuously breaks it into smaller sub-problems.

In other words, we start from the "top" — the large problem we actually want to solve — and recursively ask: ***"What smaller problems are needed to solve this one?"***. Those smaller problems may then break into even smaller ones, and the process continues until we eventually reach the base cases.

However, something interesting quickly appears. Many of these recursive calls repeat the exact same sub-problems again and again. So instead of recomputing them every time, Dynamic Programming stores their solutions inside a memoization table. The next time the same sub-problem appears, the algorithm simply retrieves the stored result instead of recalculating it. This dramatically improves efficiency.

Think of Memoization like solving a maze while leaving notes behind. The first time you visit a location, you calculate the answer carefully. But once the answer is known, you leave a record so you never have to solve that exact part again.

This approach is especially useful when the number of sub-problems is very large, many sub-problems overlap heavily, recursive thinking makes the problem easier to understand. As the recursion continues, the complex problem gradually becomes simpler and simpler.

However, Top-Down Dynamic Programming still relies on recursion. And recursion comes with its own cost:

- Recursive function calls introduce overhead
- Deep recursion may consume large amounts of stack memory
- Extremely deep recursion can even lead to stack overflow

So while Memoization is elegant and intuitive, it is not always the most memory-efficient approach.

Bottom-Up Approach (Tabulation)

The Bottom-Up approach, commonly called **Tabulation**, works in the exact opposite direction. Instead of starting from the final problem, we begin from the

smallest possible sub-problems and gradually build upward toward the final solution.

This means we solve the "bottom" first. Then we use those smaller solutions to construct solutions for larger and larger problems until the overall problem is solved.

- Rather than asking recursively: *"What smaller problems do I need?"*
- The Bottom-Up approach asks: *"What information do I already know... and how can I build upon it?"*

The solutions to smaller sub-problems are stored inside a table, and every larger computation reuses these previously computed values.

This approach is often more efficient because small reusable results are computed first, no recursive calls are needed, the call stack is avoided entirely. As a result, time overhead from recursion is reduced, memory usage in the stack decreases, stack overflow problems are avoided

This makes Bottom-Up Dynamic Programming extremely popular in practice. In many optimization problems, Tabulation is considered more efficient because it carefully constructs the solution layer by layer, almost like building a structure brick by brick.

- So while Memoization feels like: *"Break the large problem downward until it becomes solvable."*
- Tabulation feels like: *"Construct the solution upward from the smallest known truths."*

Both approaches solve the same problem. But they think in opposite directions.

4. Characteristice of Dynamic Programming

Dynamic Programming is most effective when a problem contains two very important characteristics: **Optimal Substructure** and **Overlapping Subproblems**. Without these properties, Dynamic Programming usually does not work well.

Optimal Substructure

A problem has **Optimal Substructure** when the optimal solution of the larger problem can be constructed from the optimal solutions of its smaller sub-problems.

This idea is deeply connected to the Principle of Optimality. In simple terms:

If the smaller parts are solved optimally, they can contribute toward the optimal solution of the whole problem.

Recursion often helps reveal these relationships because recursive definitions naturally show how large problems depend on smaller ones.

Consider the shortest path problem in a weighted graph.

Suppose we want to find the minimum-cost path from a source node to a destination node.

That large problem can often be broken into smaller problems such as:

- Minimum-cost path from source $\rightarrow$ intermediate node
- Minimum-cost path from intermediate node $\rightarrow$ destination

If those smaller paths are already optimal, then combining them helps construct the optimal overall path.

This is the essence of Optimal Substructure.

Large optimal solutions emerge from smaller optimal solutions.

Overlapping Subproblems

The second important characteristic is **Overlapping Subproblems**. This occurs when the same smaller problems are solved repeatedly in different parts of the computation.

Instead of recalculating them every time, Dynamic Programming stores their solutions so they can be reused later. This is where DP gains most of its efficiency.

A classic example is the Fibonacci sequence. To compute: $F(n)$, we recursively compute: $F(n - 1)$ and $F(n - 2)$

But those problems break further into: $F(n - 2)$, $F(n - 3)$, $F(n - 4)$. And very quickly, the same Fibonacci values appear repeatedly. For example:

$F(n - 2)$ may be calculated multiple times unnecessarily. These repeated computations are overlapping sub-problems.

Dynamic Programming avoids this waste by storing previous results in tables and reusing them whenever needed. Instead of solving the same work repeatedly, the algorithm remembers. And that memory is what gives Dynamic Programming its power.

5. Applications of Dynamic Programming

Dynamic Programming has an enormous range of applications across Computer Science and Mathematics. Some common applications include:

- **Optimization Problems:** Knapsack Problem, Shortest Path Problems, Maximum Subarray Problem
- **Computer Science Problems:** Longest Common Subsequence, Edit Distance, String Matching
- **Operations Research:** Inventory Management, Scheduling Problems, Resource Allocation

In many of these problems, brute force solutions are simply too expensive. Dynamic Programming transforms them into something computationally manageable.

6. Advantages of Dynamic Programming

Dynamic Programming offers several important advantages.

- **Simplifies Complex Problems:** DP breaks difficult problems into smaller, manageable sub-problems. Instead of solving one enormous problem directly, we solve many smaller ones step by step. This greatly simplifies problem-solving.
- **Improves Efficiency:** Because sub-problem solutions are reused instead of recalculated repeatedly, Dynamic Programming can drastically reduce time complexity. What would normally take exponential time can often become polynomial time. That improvement can be massive.
- **Finds Optimal Solutions:** Dynamic Programming is especially powerful for optimization problems because it systematically explores valid possibilities

and preserves the best solutions. This makes it highly reliable for finding optimal answers.

- **Highly Versatile:** DP can be applied to many categories of problems: Optimization; Decision making; Sequences; Graph algorithms; Resource management. Its flexibility makes it one of the most important techniques in Computer Science.

7. Disadvantages of Dynamic Programming

Despite its power, Dynamic Programming also comes with several disadvantages.

- **High Memory Usage:** DP often requires storing many intermediate results. For large problems, these tables can consume significant memory. Sometimes memory becomes the biggest limitation.
- **Difficult to Design:** Dynamic Programming is not always easy to implement. The programmer must carefully identify the sub-problems, define recurrence relations, determine dependencies, decide storage structures. This requires strong analytical thinking and problem-solving skills.
- **High Conceptual Complexity:** Although DP is powerful, many programmers avoid it because simpler approaches are often easier to implement and debug. Techniques like Greedy Algorithms; Backtracking; Divide and Conquer are sometimes preferred when possible. Dynamic Programming can feel mathematically heavy compared to these alternatives.
- **Sensitive to Constraint Changes:** Dynamic Programming solutions are often tightly connected to the problem constraints. Even small changes in the problem conditions may require redesigning the recurrence relation, modifying the state representation, rebuilding the entire DP structure. This makes DP solutions less flexible in some situations. And because of that, designing a good Dynamic Programming solution is not only about coding. It is about deeply understanding the structure of the problem itself.

Summary

Dynamic Programming is one of the most powerful optimization techniques in Computer Science. Instead of repeatedly solving the same computations,

Dynamic Programming breaks a complex problem into smaller overlapping sub-problems, solves them efficiently, stores their results, and reuses them whenever needed.

At its core, DP is built upon two major ideas:

- **Optimal Substructure** → optimal solutions can be constructed from smaller optimal solutions.
- **Overlapping Subproblems** → the same smaller problems appear repeatedly during computation.

Because of these characteristics, Dynamic Programming can transform problems that are computationally expensive into problems that are much more manageable.

We also explored the two major approaches of Dynamic Programming:

- **Top-Down (Memoization)** → solve recursively while storing repeated results.
- **Bottom-Up (Tabulation)** → build solutions iteratively from the smallest sub-problems upward.

Although both approaches solve the same problem, they differ in philosophy, implementation, and efficiency considerations.

We then examined where Dynamic Programming is commonly applied, ranging from optimization problems, graph algorithms, string processing, scheduling and resource allocation, to many other areas of Computer Science and Operations Research.

At the same time, Dynamic Programming is not always simple. It often requires careful analysis, strong understanding of recursion, well-defined recurrence relations, and efficient state management

And while it can dramatically improve time complexity, it may also consume large amounts of memory and become difficult to design correctly.

References

<https://www.geeksforgeeks.org/dsa/dynamic-programming/>

https://cp-algorithms.com/dynamic_programming/intro-to-dp.html

<https://www.youtube.com/watch?v=5dRGRueKU3M&list=PLJULIIVhz0rE83NKhng7acXYIeA0o1dXb&index=1>
<https://www.youtube.com/watch?v=Hdr64IKQ3e4>

In the following subsections, we will explore some of the most common Dynamic Programming problems—some of which we may revisit from earlier topics—to better understand how Dynamic Programming actually works in practice.

Because understanding DP is not only about memorizing formulas or recurrence relations. It is about learning how to think recursively, identify reusable structure, and transform repeated computation into efficient computation. And once you begin to see those patterns, many difficult problems suddenly become solvable.

- 🍷 [4.1. 0/1 Knapsack Problem](#)
- 🍷 [4.2. All Pairs Shortest Path](#)
- 🍷 [4.3. Longest Common Subsequence](#)

Next Section:

- 🍷 [5. Recursion](#)